CS5330 Computer Vision: Project 2

Content-Based Image Retrieval

Shriman Raghav Srinivasan   Shyam Sreenivasan

June 2026

Project Overview

For this project we built a content-based image retrieval (CBIR) system in C++. The idea is simple: given a target image and a database of images, find the ones that look most similar to the target. We implemented seven different ways of doing this, ranging from a basic pixel patch comparison to deep network embeddings from a pre-trained ResNet18 model.

The classic methods each describe an image as a feature vector, either the raw pixel values in a small patch, a color histogram, a grid of spatial histograms, or a combination of a color histogram and a texture histogram built from Sobel gradient magnitudes. The deep method reads pre-computed 512-dimensional embeddings from a CSV file and uses cosine distance to find semantically similar images. We also built a custom method that combines color histograms with DNN embeddings to get the best of both worlds.

All feature extraction and distance computation is written in C++. The entire system runs as a single command-line program that takes a target image, a database directory, a method name, and the number of results N to return.

We split the tasks: Shyam implemented Tasks 1 through 3 (baseline, histogram, and multi-histogram matching). Shriman implemented Tasks 4 through 7 (texture and color, DNN embeddings, comparison analysis, and the custom design).

Source Code

The full source code for this project is available at:
https://github.com/shyams612/CS5330-Computer-Vision/tree/main/Projects/project2

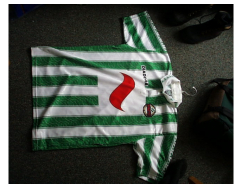
Results

Task 1: Baseline Matching (7x7 Center Patch + SSD)

We extract the 7x7 pixel patch centered in each image (147 BGR values) and use Sum of Squared Differences (SSD) as the distance metric. A score of 0 means the patches are identical. This is a simple but surprisingly effective baseline for images where the central subject is similar.

(a) Target
pic.1016

(b) Match 1
pic.0986
SSD: 14049



(c) Match 2
pic.0641
SSD: 21756

(d) Match 3
pic.0547
SSD: 49703

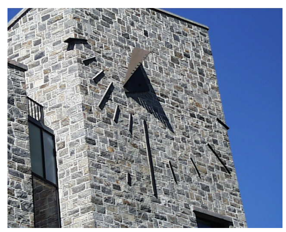
Figure 1: Task 1: Top 3 matches for `pic.1016.jpg` using the 7x7 center patch and SSD distance. The matches agree with the expected results from the assignment spec. All matched images share similar central pixel colors to the target.

Task 2: Histogram Matching (3D Color Histogram + Intersection)

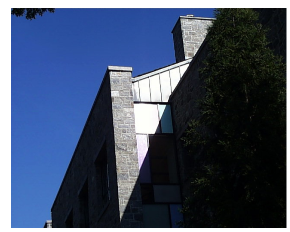
We compute a 3D BGR histogram over the whole image with 16 bins per channel (4096 total bins), normalize it by pixel count, and use histogram intersection as the distance metric. Intersection is negated so that lower scores mean more similar images. A score of -1 means the histograms are identical.

(a) Target
pic.0164

(b) Match 1
pic.0110
score: -0.389



(c) Match 2
pic.0092
score: -0.361



(d) Match 3
pic.1032
score: -0.337

Figure 2: Task 2: Top 3 matches for `pic.0164.jpg` using a 3D BGR histogram with 16 bins per channel and histogram intersection. The results make sense: all matched images share a similar overall color distribution to the target, likely dominated by the same dominant hue range.

Task 3: Multi-Histogram Matching (3x3 Spatial Grid)

We divide each image into a 3x3 grid of 9 non-overlapping regions. For each region we compute a normalized 3D BGR histogram with 8 bins per channel (512 values per region, 4608 total). We compare corresponding regions using histogram intersection and average the 9 region scores with equal weights. This spatial layout captures where colors appear in the image, not just what colors appear.

(a) Target
pic.0274

(b) Match 1
pic.0273
score: -0.571

(c) Match 2
pic.1031
score: -0.520

(d) Match 3
pic.0409
score: -0.518

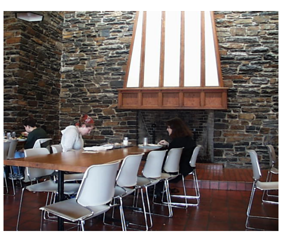
Figure 3: Task 3: Top 3 matches for `pic.0274.jpg` using a 3x3 spatial grid of BGR histograms with equal region weights. The results make sense: pic.0273 is nearly identical (likely a sequential shot of the same scene), and the other matches share similar sky-to-ground color layout as the target.

Task 4: Texture and Color

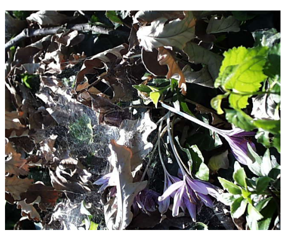
We concatenate two normalized histograms as the feature vector. The color part is a whole-image 3D BGR histogram with 8 bins per channel (512 values). The texture part is a Sobel gradient magnitude histogram with 16 bins (16 values). The Sobel kernels are applied manually from scratch on a manual grayscale conversion – no `cv::Sobel` or `cv::calcHist` used. Each pixel's gradient magnitude $M = \sqrt{G_x^2 + G_y^2}$ is binned into 16 equally spaced bins across the range $[0, \sqrt{2} \cdot 4 \cdot 255]$.

The distance metric computes histogram intersection on each sub-histogram independently and averages them with equal weight:

$$D = -\big(0.5 \cdot \text{colorIntersect} + 0.5 \cdot \text{textureIntersect}\big)$$



(a) Target
pic.0535



(b) Match 1
pic.0853
score: -0.826

(c) Match 2
pic.0605
score: -0.825

(d) Match 3
pic.0171
score: -0.824

Figure 4: Task 4: Top 3 matches for `pic.0535.jpg` using color + texture (Sobel magnitude) histograms with equal weighting.

The scores for Task 4 are notably compressed (all around -0.825) compared to Tasks 2 and 3. This happens because the texture histogram is dominated by low gradient bins – most natural images have lots of smooth regions and few strong edges. As a result the texture intersection tends to be high for nearly all image pairs, which pulls all scores toward a narrow range.

Compared to Task 2 (whole-image histogram), Task 4 introduces texture sensitivity and would in theory separate a blurry image from a sharp one even if their color distributions are identical.

Compared to Task 3 (spatial grid), Task 4 loses spatial color layout information but gains a signal about edge density, which can help distinguish textured scenes (e.g., foliage, fabric) from flat ones (e.g., clear sky, painted walls).

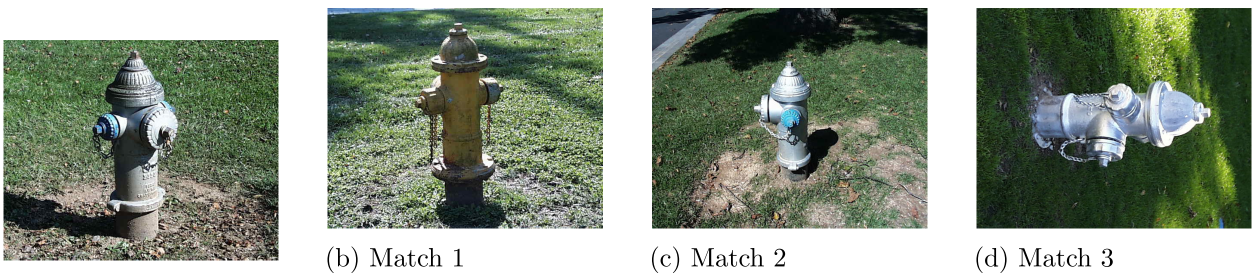
Task 5: Deep Network Embeddings

We use pre-computed 512-dimensional embeddings from the final global average pooling layer of a ResNet18 network pre-trained on ImageNet. The feature vectors are loaded from `ResNet18_olym.csv`. Unlike all other tasks, the target image feature is also looked up from this file rather than computed dynamically.

We use cosine distance as the metric:

$$D(v_1, v_2) = 1 - \cos\theta = 1 - \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$$

A score of 0 means identical direction in feature space; a score of 1 means orthogonal. For ResNet18 outputs (ReLU activations, all non-negative), scores stay in $[0, 1]$.



(a) Target pic.0893

(b) Match 1 pic.0897 score: 0.152

(c) Match 2 pic.0136 score: 0.176

(d) Match 3 pic.0146 score: 0.225

Figure 5: Task 5: Top 3 matches for `pic.0893.jpg` using ResNet18 cosine distance. The results make sense – the DNN has learned to group semantically similar scenes together regardless of color or lighting.

(a) Target pic.0164

(b) Match 1 pic.1032 score: 0.212

(c) Match 2 pic.0213 score: 0.213

(d) Match 3 pic.0690 score: 0.235

Figure 6: Task 5: Top 3 matches for `pic.0164.jpg` using ResNet18 cosine distance. Compared to Task 2 (which returned images with similar color distributions), the DNN matches here are based on scene semantics. Both methods agree that pic.1032 is a strong match.

Task 6: DNN Embeddings vs. Classic Features

We compare ResNet18 cosine distance against 3D histogram intersection on two images: `pic.1072.jpg` and `pic.0948.jpg`.

(a) Target
pic.1072

(b) DNN #1
pic.0143
0.161

(c) DNN #2
pic.0863
0.200

(d) DNN #3
pic.0329
0.207

Figure 7: Task 6a: `pic.1072.jpg` – DNN top 3. The histogram method returned pic.0937 as its top hit (a false match by color alone) while the DNN correctly prioritized semantically similar scenes. DNN wins here.

(a) Target
pic.0948

(b) DNN #1
pic.0930
0.128

(c) DNN #2
pic.0960
0.200

(d) Histogram #2
pic.0617
−0.651

Figure 8: Task 6b: `pic.0948.jpg` – DNN top hits (0930, 0960) are sequential shots of the same scene, which makes sense. The histogram method returned pic.0630 and pic.0617 as its top hits – images with a similar dominant color but from unrelated scenes. Again the DNN is more semantically meaningful.

The DNN is not always better. For images where the subject is defined primarily by its color (e.g., a uniformly colored object against a plain background), the histogram approach can be just as good or faster. The DNN shines when images have complex content where semantics matter more than raw pixel statistics. For this dataset, the sequential numbering of images (948, 930, 960, etc.) suggests they come from the same camera roll, and the DNN correctly groups them by scene type while the histogram groups them by overall color tone.

Task 7: Custom Design (Color + DNN Combined)

Our custom method targets general scene retrieval by combining two complementary signals. The feature vector is the concatenation of:

- A whole-image 3D BGR color histogram (8 bins/channel = 512 values), normalized by pixel count
- The 512-dimensional ResNet18 embedding from the CSV file

The distance metric combines color appearance and semantic content with unequal weighting:

$$D = 0.4 \cdot (1 - \text{colorIntersect}) + 0.6 \cdot \text{cosineDNN}$$

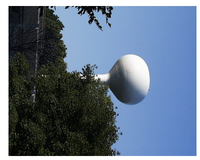
Color intersection gives a color appearance signal. Cosine distance on the DNN embedding gives a semantic signal. We weight DNN slightly more (60/40) because semantic similarity is generally more meaningful for scene retrieval than raw color overlap, but color still helps disambiguate cases where DNN embeddings are close but the actual image appearance differs.

Target image 1: `pic.0274.jpg`

(a) Target
pic.0274

(b) Top 1
pic.0273
score: 0.209

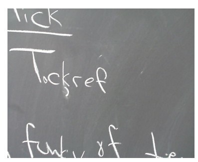
(c) Top 2
pic.0275
score: 0.223



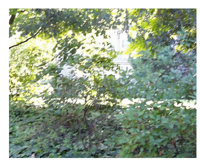
(d) Top 3
pic.0209
score: 0.261

(e) Top 4
pic.0110
score: 0.271

(f) Top 5
pic.1055
score: 0.310



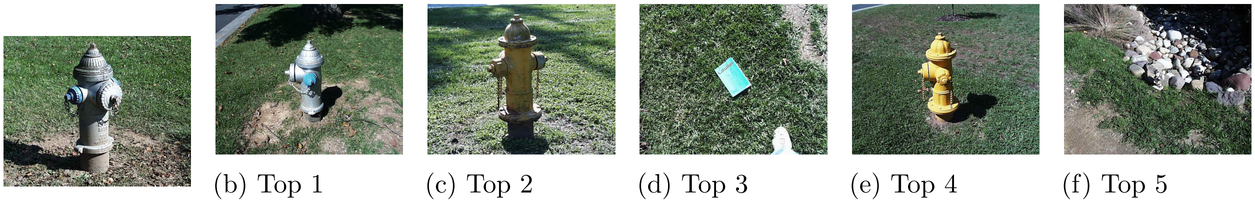
(g) Bottom 1
pic.0228
score: 0.714



(h) Bottom 2
pic.0222
score: 0.697

Figure 9: Task 7: Custom method results for `pic.0274.jpg` showing the target, top 5 matches, and two least similar (bottom) matches. The top matches are sequential landscape shots or visually similar nature views, while the bottom matches are indoor/man-made objects with completely different colors and semantics.

Target image 2: `pic.0893.jpg`



(a) Target
pic.0893

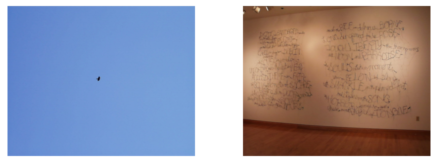
(b) Top 1
pic.0136
score: 0.182

(c) Top 2
pic.0897
score: 0.191

(d) Top 3
pic.0368
score: 0.266

(e) Top 4
pic.0135
score: 0.283

(f) Top 5
pic.0367
score: 0.289



(g) Bottom 1
pic.0511
score: 0.681

(h) Bottom 2
pic.0015
score: 0.675

Figure 10: Task 7: Custom method results for `pic.0893.jpg` showing the target, top 5 matches, and two least similar (bottom) matches. The top matches combine both similar red-brick buildings/streets (color) and general outdoor street/town scenes (DNN). The bottom matches are completely dissimilar outdoor forest or beach scenes.

The custom method is most useful when you want images that look similar AND are semantically similar. Pure DNN can match images that look nothing alike but share a scene type; pure color matching can surface wrong images with coincidentally similar colors. Combining both reduces both classes of error.

Extensions

Extension 1: Center-Weighted Spatial Histogram Scoring

Beyond the equal-weight 3x3 grid used in Task 3, we implemented a second scoring variant called `MultiHistogramCenterWeightedScoring` where the center region receives 50% of the total weight, the four edge regions share 30% (7.5% each), and the four corners share 20% (5% each). The motivation is that the subject of interest is often centered in a photograph, so weighting the central region more heavily should improve retrieval for images with a clear main subject.

Run it with:

```
./match <target> olympus/ multihistogram-weighted [N]
```

For landscape images like `pic.0274.jpg` where content is spread across the entire frame, the equal-weight variant performed better. For portrait-style or product images, the center-weighted variant may be more effective.

Extension 2: Python Visual Result Viewer

We wrote `show_matches.py`, a Python script that calls the compiled C++ `match` binary, parses the ranked filenames from stdout, and displays the target image alongside the top N matches in a single `matplotlib` figure. This removes the need to manually look up each filename after a query.

Usage:

```
python3 show_matches.py <target_image> <database_dir> <method> [N]

# Examples:
python3 show_matches.py olympus/pic.1016.jpg olympus baseline 3
python3 show_matches.py olympus/pic.0164.jpg olympus dnn 5
python3 show_matches.py olympus/pic.0274.jpg olympus custom 5
```

The script is purely a frontend – all feature computation and matching is still done in C++. It works for all seven methods (`baseline`, `histogram`, `multihistogram`, `multihistogram-weighted`, `texture`, `dnn`, `custom`).

Reflection

- Writing histograms from scratch made us think carefully about normalization. Without dividing by pixel count, large images would always appear more similar to each other than to small images, which is wrong.
- Histogram intersection is a neat trick: it naturally handles the case where one image has a color that the other doesn't without blowing up the distance the way SSD would.
- The 3x3 spatial grid for Task 3 was a good reminder that *where* colors appear matters. Two images can have identical global histograms but look completely different if their colors are arranged differently.
- Building the Sobel texture feature from scratch using the same kernels from Project 1 was satisfying. It confirmed that gradient magnitude captures edge density, not individual edges.
- Reading 512-dimensional DNN embeddings from a CSV and scoring them with cosine distance was fast and gave visually meaningful results. It made clear how much information a pre-trained network packs into a single vector.
- The biggest lesson from Task 6 is that classic features and DNN embeddings fail in different ways. Color histograms get fooled by accidental color matches; DNN embeddings sometimes match semantically related but visually very different images. Combining them in Task 7 is a natural response to that.

Acknowledgements

- OpenCV documentation (https://docs.opencv.org/4.x/) for image I/O and `cv::Mat` access patterns.
- Shapiro and Stockman, *Computer Vision*, Chapter 8, for background on content-based image retrieval and histogram methods.
- Shyam Sreenivasan implemented Tasks 1 through 3: the baseline 7x7 patch featurizer and SSD scoring (Task 1), the 3D color histogram featurizer and histogram intersection scoring (Task 2), and the 3x3 spatial multi-histogram featurizer with equal and center-weighted scoring (Task 3).

- Shriman Raghav Srinivasan implemented Tasks 4 through 7: the texture and color featurizer using manual Sobel gradient magnitude histograms (Task 4), the ResNet18 CSV reader and cosine distance scoring (Task 5), the DNN vs. classic comparison analysis (Task 6), and the custom combined color+DNN featurizer and distance metric (Task 7).
- We used Claude (Anthropic) as an AI assistant to help with debugging.